rubyopt
: Prevents Ruby from examining the RUBYOPT environment variable. You should probably disable this in an environment you want to secure. Enabled by default.

yjit
: Enables the yjit compiler. Disabled by default.

Making Your Code an Executable Program

It is a little clunky to have to call `ruby my_code.rb` when you want to run your code; it'd be easier to just be able to say `my_code.rb`. This is more of an operating system tip than a Ruby tip, but on Unix systems, this is doable with just a couple of steps.

First, you need to make the file executable by changing the mode of the file. To oversimplify slightly, the mode of the file is metadata that determines if the current user can read from, write to, or execute a given file. Typically on a Unix-based system, the command to make a file executable is `chmod +x <FILENAME>`. The `chmod` command is Unix-speak for "change the mode of the file" and `+x` means "make it executable", and the FILENAME is the filename. For more on the Unix command line, see *Appendix 3, Command-Line Basics*.

Having made the file executable, we also need to tell Unix what it means for the file to be executable. For a Ruby script, what we mean is "run this file through the Ruby runtime". And we tell that to the Unix system through a special comment on the first line of the file that starts with `#!` and contains the name of an interpreter program that should be used to run the file. (This comment is often referred to as "shebang" because the two characters that make it up are the `#`, which is a musical sharp, and `!`, which coders often call "bang".)

In most cases, the shebang command you use to invoke Ruby can look like this:

```
#!/usr/bin/env ruby

# Ruby code goes here
```

The path `/usr/bin/env` is, for weird Unix reasons, a common cross-platform way to ensure you are running the proper shell and `ruby` is the Ruby interpreter. (There are other ways to specify the Ruby interpreter, but this is the most recommended way to have the script run in most common Unix setups.)

Anyway, once you've done both these steps, you can invoke your script directly:

```
$ my_code.rb
```

If for some reason you want to send command-line options to Ruby itself (for example, you might want to run in warning mode), you can do so by setting an environment variable called `RUBYOPTS`, as in

```
RUBYOPTS="-w" my_code.rb
```

The Ruby interpreter will, by default, look in that environment variable for options before it starts.

Any arguments after the file name are passed to the Ruby code itself, so this would be a good time to show how to access those arguments…